Evaluating agents

Architecture 4: University teaching deck

How do we know that an agent works, how well, how reliably, and how safely?

Public benchmarks, reading scores critically, outcome and trajectory evaluation, LLM-as-a-judge, rubrics, pass@k, pass^k, safety, and custom evals.

Based on An Illustrated Guide to AI Agents
Maarten Grootendorst and Jay Alammar (O'Reilly, 2026)

Learning outcomes and conceptual route

Benchmarks

Describe what a benchmark is and the main categories of agent benchmark. Apply a list of questions to read a benchmark table critically.

Outcome and trajectory

Compare human evaluation, exact match, programmatic checks, LLM-as-a-judge, and rubrics. Explain what trajectory evaluation reveals that outcome evaluation misses.

Reliability, safety, and your own evals

Distinguish pass@k from pass^k. Organize safety tests by threat model. Outline how to start a small custom evaluation and run it on every change.

Source: Ch. 7, PDF pp. 347–379

From the output to the trajectory

The natural next question

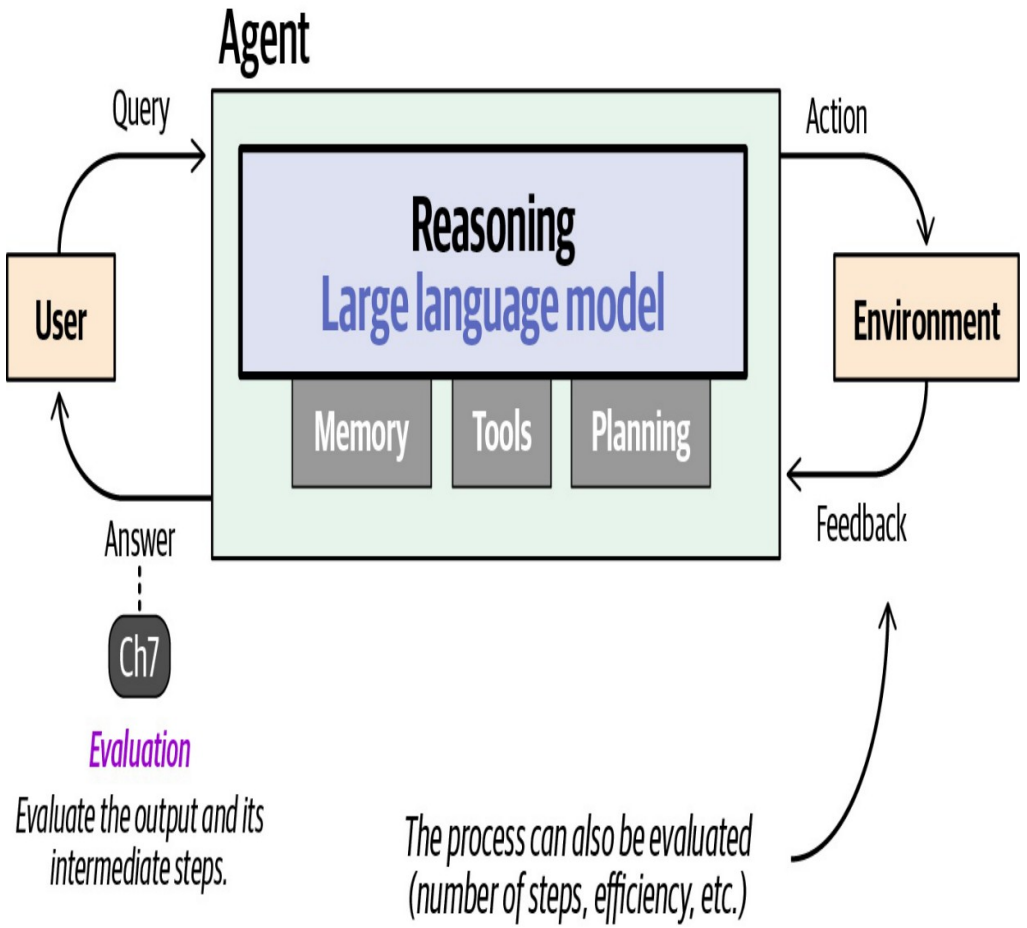
The previous chapters built reasoning, tools, memory, and planning. The question now is how we know whether any of it works.

Evidence, not impression

Evaluation moves you from "it seems to work" to "I have evidence it works", and helps you catch the moment it stops working.

Two targets

The figure marks the output (the answer) and the process (steps, efficiency, intermediate steps), which is the agent's trajectory.



Grootendorst & Alammar (2026). Fig. 7-1, PDF p. 348

Evaluation is an underinvested area

A common pattern

Teams often rely on manual testing and intuition longer than they should. The book calls evaluations one of the most underinvested areas in agent development.

The cost of intuition

Without evaluations, teams cannot ship improvements confidently, and they struggle to diagnose regressions when something that worked stops working.

The plan of the chapter

Start with benchmarks, which you meet in model announcements and research papers. Then build toward the tools and concepts needed to design your own evaluations.

What a benchmark is

Fixed tasks and a score

A benchmark is a fixed set of tasks paired with a way to score them, so that different models are measured against the same yardstick.

SWE-bench Verified

Real GitHub issues from open source projects. The agent reads the issue, explores the repository, and writes a patch that passes hidden tests. The score is the share solved.

A typical table

The figure is a benchmark table from a recent model release: many benchmarks in rows, models in columns, one number per cell.

Benchmark		Gemini 3.1 Pro Thinking (High)	Gemini 3 Pro Thinking (High)	Sonnet 4.6 Thinking (Max)	Opus 4.6 Thinking (Max)	GPT-5.2 Thinking (xhigh)	GPT-5.3-Codex Thinking (xhigh)
Humanity's Last Exam Academic reasoning (full set, text + MM)	No tools	**44.4%**	37.5%	33.2%	40.0%	34.5%	–
	Search (blocklist) + Code	51.4%	45.8%	49.0%	**53.1%**	45.5%	–
ARC-AGI-2 Abstract reasoning puzzles	ARC Prize Verified	**77.1%**	31.1%	58.3%	68.8%	52.9%	–
GPQA Diamond Scientific knowledge	No tools	**94.3%**	91.9%	89.9%	91.3%	92.4%	–
Terminal-Bench 2.0 Agentic terminal coding	Terminus-2 harness	**68.5%**	56.9%	59.1%	65.4%	54.0%	64.7%
	Other best self-reported harness	–	–	–	–	62.2% (Codex)	**77.3%** (Codex)
SWE-Bench Verified Agentic coding	Single attempt	80.6%	76.2%	79.6%	**80.8%**	80.0%	–
SWE-Bench Pro (Public) Diverse agentic coding tasks	Single attempt	54.2%	43.3%	–	–	55.6%	**56.8%**
LiveCodeBench Pro Competitive coding problems from Codeforces, ICPC, and IOI	Elo	**2887**	2439	–	–	2393	–

Grootendorst & Alammar (2026). Fig. 7-2, PDF p. 350

Why one number needs care

Different claims

An 80% on SWE-bench Verified and an 80% on MMLU are different claims about different capabilities, measured in different ways.

Different runs

Some scores come from one run, while others aggregate many runs, sometimes in ways that could be deceptive.

Different scorers

Some use deterministic test suites. Others rely on an LLM judge that introduces its own biases.

Not always agentic

Some benchmarks in these tables do not measure agent capability at all, only the non-agentic capabilities of the underlying LLM.

Coding and software engineering benchmarks

Most mature family

Coding benchmarks became a gold standard because code is verifiable. It either executes and passes the unit tests or it does not.

SWE-bench

Each example is a real GitHub issue that the agent resolves with a patch editing multiple files. Best agents resolved about 2% in 2023. By 2026, leading agents passed 70% on SWE-bench Verified, a curated version with ambiguous or unreliable test cases removed.

Terminal-Bench

The agent operates a Linux terminal to solve software, system, and environment problems. Coding agents and their evaluation return in Chapter 10.

Tool use and computer-use benchmarks

BFCL
The Berkeley Function Calling Leaderboard tests function-calling accuracy across hundreds of API schemas, including nested parameters and ambiguous inputs.

τ-bench
Goes deeper with realistic multi-turn customer service scenarios that require following business policies and keeping context over a long conversation.

OSWorld
Places agents in simulated computer environments to edit documents, manage files, or configure software through graphical interfaces.

WebArena
Narrows the scope to browser tasks in realistic replicas of live websites, such as a Reddit forum or a GitLab instance.

Real-world, reasoning, and specialized benchmarks

Real-world tasks

GAIA poses real-world questions that need files and web search. GDPval draws on knowledge work in many professions. Benchmarks cluster around programming while most occupations go untested.

Reasoning and knowledge

MMLU spans dozens of subjects. GPQA Diamond uses graduate-level questions that are hard even for experts. Humanity's Last Exam is harder still.

A caution

Reasoning and knowledge benchmarks often test the underlying LLM, not the agentic ability of using tools across multiple steps.

Specialized

ARC-AGI for abstract puzzles, MMMU for multimodal reasoning, and needle-in-a-haystack tests for long-context retrieval.

Grootendorst & Alammar (2026). PDF pp. 351–352

Benchmark categories at a glance

Coding and tool use

SWE-bench Verified and Terminal-Bench measure patch correctness and test passage. BFCL and τ-bench measure function-calling accuracy and multi-turn policy following.

Computer use and real-world

OSWorld and WebArena measure action sequences in desktop and browser environments. GAIA and GDPval measure reasoning and retrieval on open-ended knowledge tasks.

Reasoning and knowledge

MMLU, GPQA Diamond, and Humanity's Last Exam measure exam-style questions across domains.

Specialized

ARC-AGI measures abstract reasoning, MMMU multimodal reasoning, and needle-in-a-haystack tests long-context retrieval.

Grootendorst & Alammar (2026). Table 7-1, PDF pp. 352–353

Reading scores critically: how, partial credit, harness, runs

How was it produced?

Different evaluation setups give different results on the same benchmark. Deterministic checks such as unit tests differ from LLM judges.

Partial success?

Some benchmarks are all-or-nothing, others give partial credit. A high partial-credit score can hide an agent that rarely finishes a task end to end.

Which harness?

Prompts, tool-call parsing, retries, and context handling move the number even with the model fixed. Read a score as a model-and-harness pair.

Single run or many?

One run can flatter a lucky result. More trustworthy results aggregate many runs with a variance estimate, such as pass@k.

Reading scores critically: contamination and tuning

In the training data?

If a model trained on examples from the benchmark, memorization skews the score. This is hard to verify except for fully open models that disclose their data.

Who ran it?

Self-reported scores deserve more scrutiny than third-party replications. Even independent leaderboards can be gamed, as The Leaderboard Illusion describes.

Prompts tuned?

Prompt engineering against a known benchmark can move scores without any generalizable improvement. The same approach must apply to every system compared.

Does the scaffolding generalize?

A retrieval pipeline, custom tool, or verification step may help the score while needing heavy engineering to transfer to your own deployment.

Reading scores critically: saturation, variant, cost

Is it saturated?
Once leading models exceed a threshold, say 85%, score differences become noisy and less indicative of real performance. The industry then moves to a harder benchmark or variant.

Which subset or variant?
Many benchmarks have versions of different difficulty, such as GPQA versus GPQA Diamond, ARC Easy versus ARC Challenge, and MMLU versus MMLU-Pro.

What was the cost?
A system that scores 20% more while spending ten times the tokens or time is not straightforwardly better. Cost, token count, and latency matter more in production than in an evaluation setup.

Outcome versus trajectory evaluation

Outcome evaluation

Scores the final result of a task, such as the patch produced, the answer returned, or the report written, without regard to how the agent got there.

Trajectory evaluation

Examines the sequence of steps and tool calls the agent made along the way. The chapter treats it in a smaller later section.

Why both

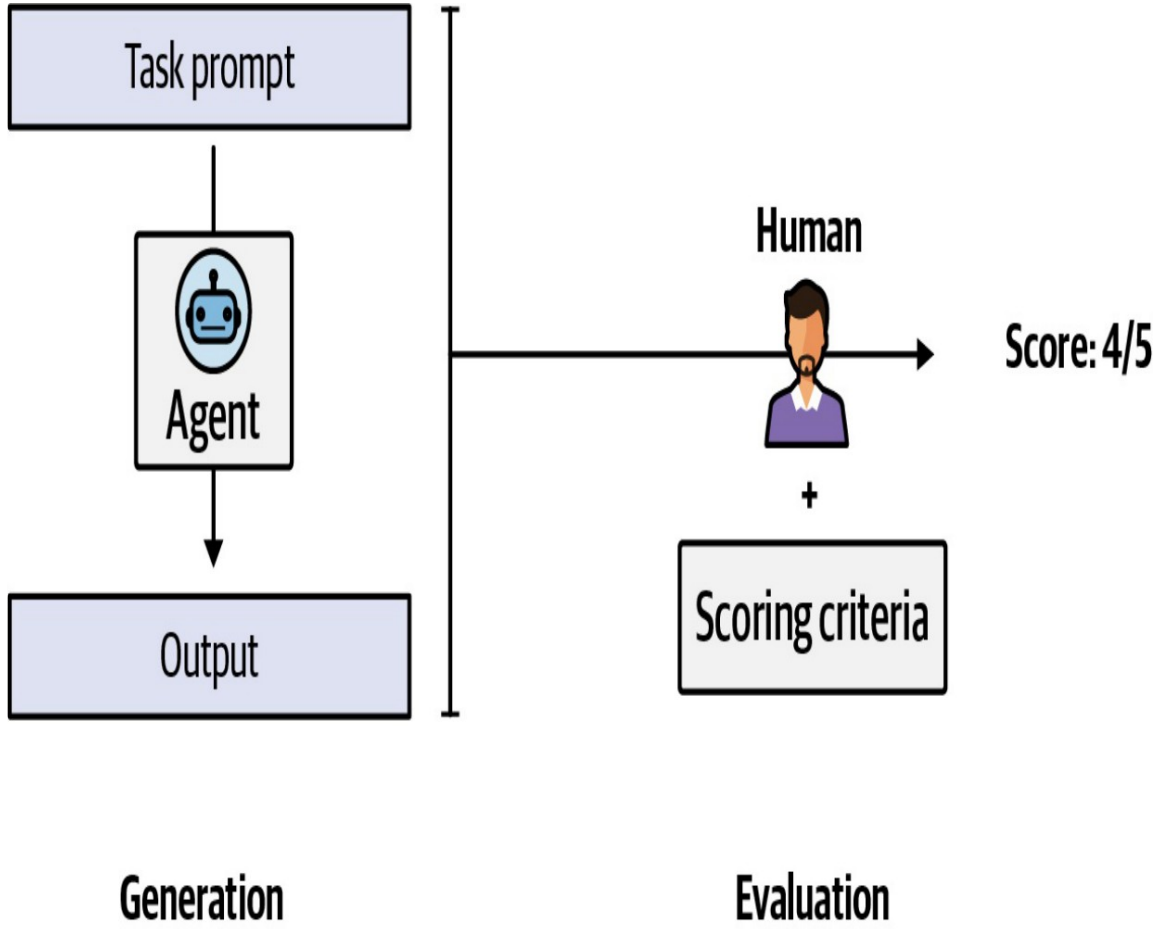
Outcome tells you whether the agent succeeded. Trajectory tells you how it got there or how it failed. Benchmarks use a number of scoring methods, which the next slides cover.

Human evaluation: direct scoring

A person judges
The most direct way to score an agent's output is to have a person read it and judge it against criteria, for example giving 4 out of 5.

Needs clear criteria
It works well when quality has an absolute sense and evaluators apply clear criteria consistently.

The difficulty
It is often hard to get an objective score in isolation. That motivates the second form of human evaluation.



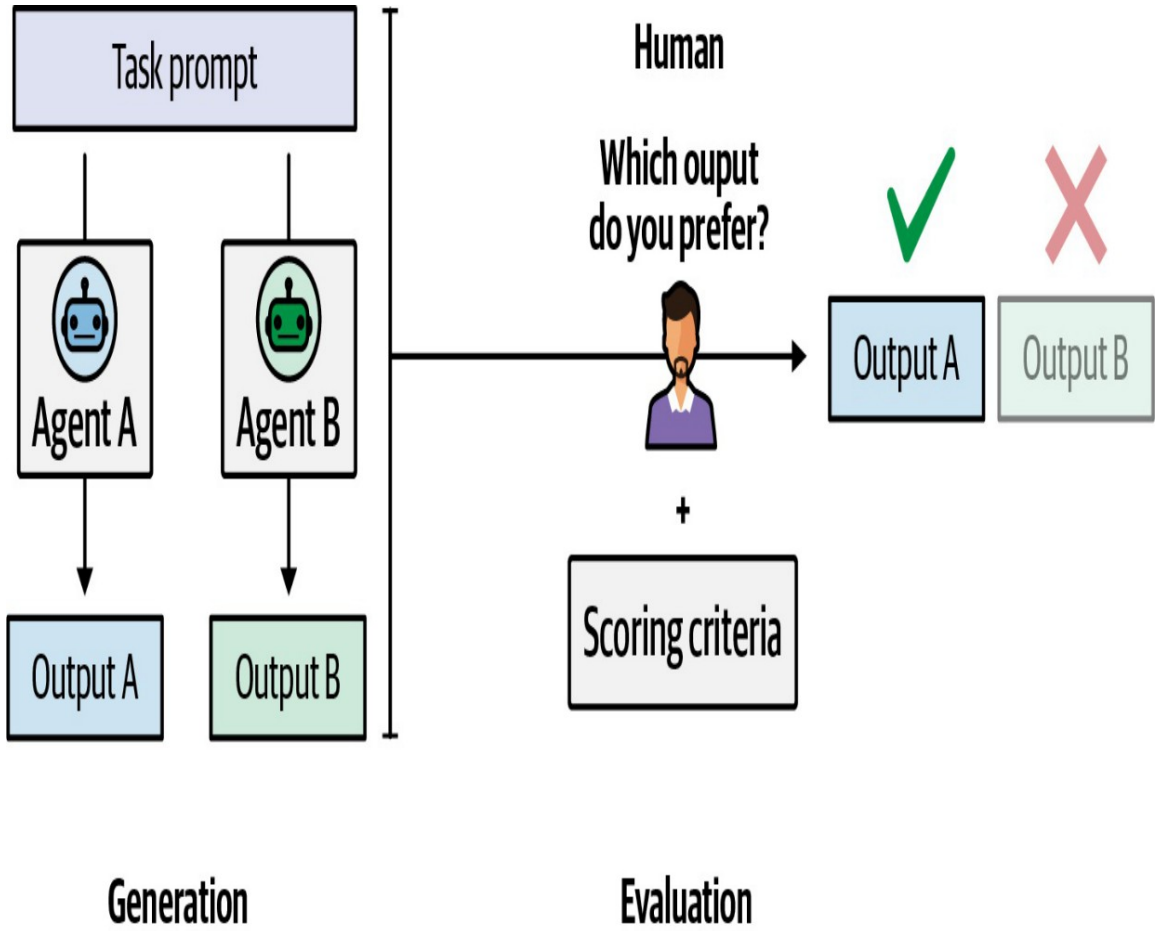
Grootendorst & Alammar (2026). Fig. 7-3, PDF p. 355

Human evaluation: preference

Side by side
Preference evaluation shows two agents' outputs for the same task and asks a simpler question: which do you prefer?

Relative is easier
Relative judgments tend to be more reliable than absolute ones. It is easier to say A is better than B than to decide A deserves 4 and B deserves 3.

Guided by criteria
The evaluator is still guided by scoring criteria, as the figure shows, but the output of the judgment is a choice.



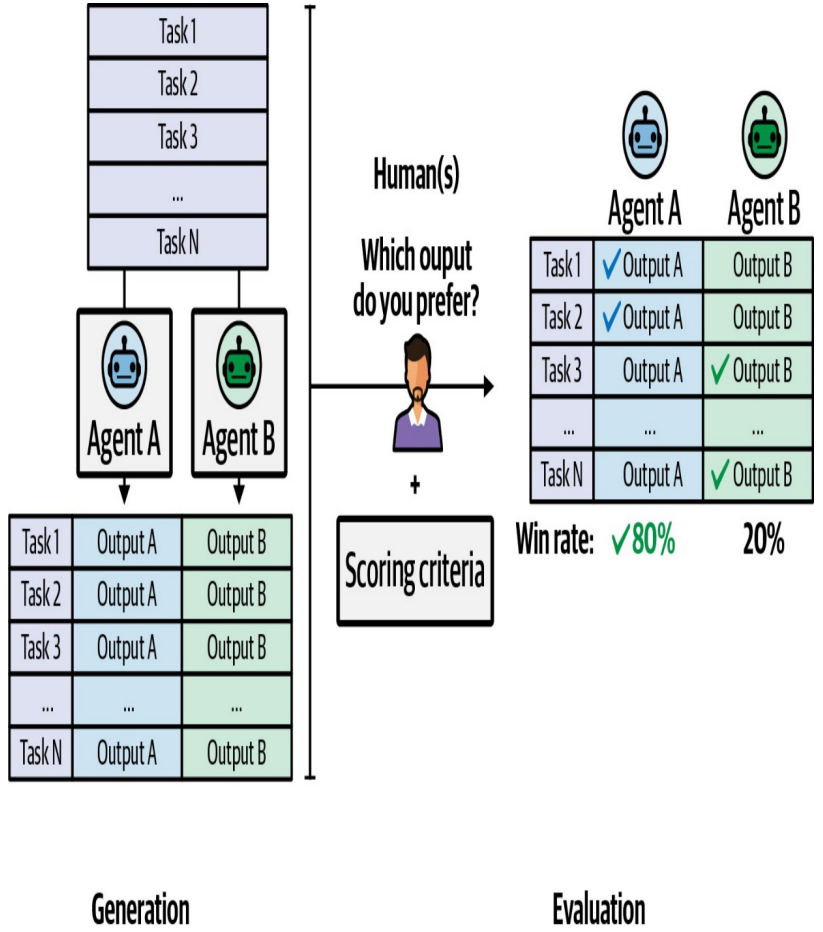
Grootendorst & Alammar (2026). Fig. 7-4, PDF p. 356

Aggregating preferences into win rates

Across a task suite
Humans compare outputs from Agent A and Agent B on every task in a suite, and the per-task preferences are aggregated.

Win rate
On the tasks in the set, Agent A was preferred on 80% and Agent B on 20%. The win rate summarizes many pairwise comparisons.

Reads as evidence
The number is only as good as the task set and the criteria, so both should be reported with it.



Grootendorst & Alammar (2026). Fig. 7-5, PDF p. 357

Elo ratings and the role of human judgment

Elo for many agents

Comparing more than two agents pairwise can use an Elo rating system, borrowed from competitive chess. Each pairwise result updates a running score of both agents, according to the strength of the opponent.

Chatbot Arena

The most prominent example at scale. Thousands of users compare anonymous model outputs, and stable rankings emerge over time using a rating similar to Elo.

The baseline

Human evaluation is slow and expensive, but all automated methods are validated against it. A new judge or metric is shown to be meaningful by correlating with human judgment.

Automated evaluation and a simple evaluator

Why automate

Builders iterate through hundreds or thousands of prompt changes, model swaps, versions, and test points. That can mean millions of complex judgments, so human evaluation does not scale.

A spectrum

Automated evaluation runs from static checks to judgments made by another model. The right method depends on the kind of output.

Benchmark object

The book defines a Benchmark with a name, a list of examples, and a scorer that returns either true or false or a score.

Evaluator object

An Evaluator creates a fresh agent for each example, runs the task, applies the scorer, and averages the passes into a pass rate.

Grootendorst & Alammar (2026). Code, PDF pp. 358–360

Exact match

A deterministic check

The output either matches the expected answer or it does not. Extracting an invoice total of $1,482.81 can be compared directly in software.

Simple but limited

Many tasks produce documents, code, or plans, where there are many valid ways to express a correct answer, so exact match breaks down.

Brittle formatting

Answers such as 1,482.81, 1482.81, or $1,482.81. with a trailing period are all flagged wrong by a strict check unless the format is pinned down.

MMLU-Pro example

A multiple-choice scorer looks for a letter from A to J. On three questions the small model scored one of three, a pass rate of 0.33.

Grootendorst & Alammar (2026). Code, PDF pp. 360–361

Programmatic checks

Automated verifiers

Instead of one string, code verifies structural properties: valid JSON, a function that passes unit tests, or, for SWE-bench, tests that fail before the patch and pass after it.

Fast, cheap, objective

This is one reason coding agents advanced faster than other kinds. The code modality allows automated checks that can also guide training.

IFEval

A benchmark of verifiable instructions, such as at most 150 words, fewer than 10 sentences, or no commas. Each instruction has its own validator.

Run on TinyAgent

Three IFEval examples were each checked by a short function. All three passed, a pass rate of 1.0.

Grootendorst & Alammar (2026). Code, PDF pp. 361–363

LLM-as-a-judge

A model scores the output

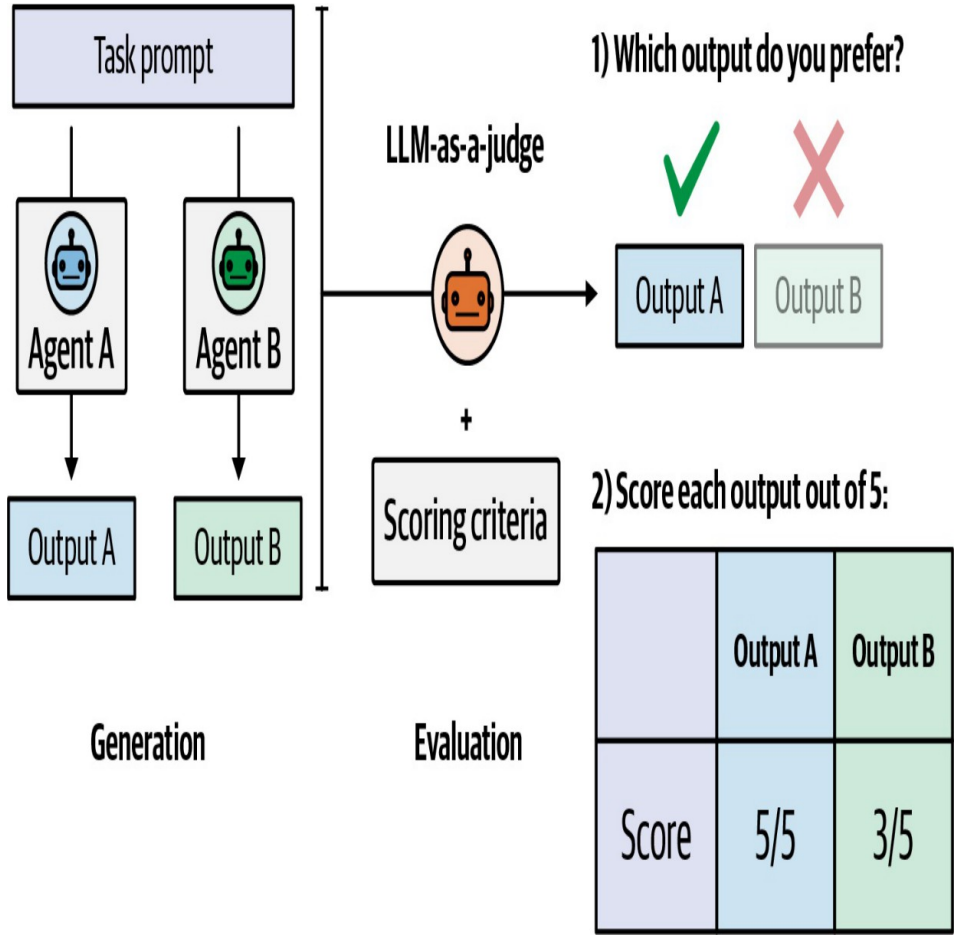
When output is too open-ended for a programmatic check, another capable model evaluates it. It can give a single score or choose a preferred output from two agents.

What it can assess

Fluency, logical coherence, consistency with a ground-truth context, and tone, which no programmatic check can do.

Failure modes

Judges can prefer confident-sounding answers, depend on presentation order, and favor their own output or their own model family.



Grootendorst & Alammar (2026). Fig. 7-6, PDF p. 364

Reducing the weaknesses of an LLM judge

Different family

Use a judge from a different model family to reduce the tendency to favor its own outputs.

Swap the order

Swap the presentation order of preference pairs across evaluation runs to control for position bias.

Ask for a rationale

Ask the judge for a written rationale, or use a reasoning model as judge, to improve consistency.

Ensembles

Use several judges that vote and aggregate their judgments.

Grootendorst & Alammar (2026). PDF p. 364

A judge scorer on TinyAgent

A different, stronger judge

The book uses a Gemini model from a free tier as the judge, so that the judge differs from the agent's own model. A locally run larger model is an alternative.

Fine-grained scores

The scorer asks for a number from 0 to 1 using a simple prompt with the expected answer and the response, instead of returning true or false.

Open-ended questions

The same MMLU-Pro questions are rewritten without options. The pass rate came out at 0.8, with one answer scored 0.4 because it was close but wrong.

A judge failure

An answer saying "happiness" received 1.0, although the expected answer was "good". LLMs judging LLMs are imperfect, so mix metrics.

Grootendorst & Alammar (2026). Code, PDF pp. 364–367

Rubric-based evaluation

A structured rubric

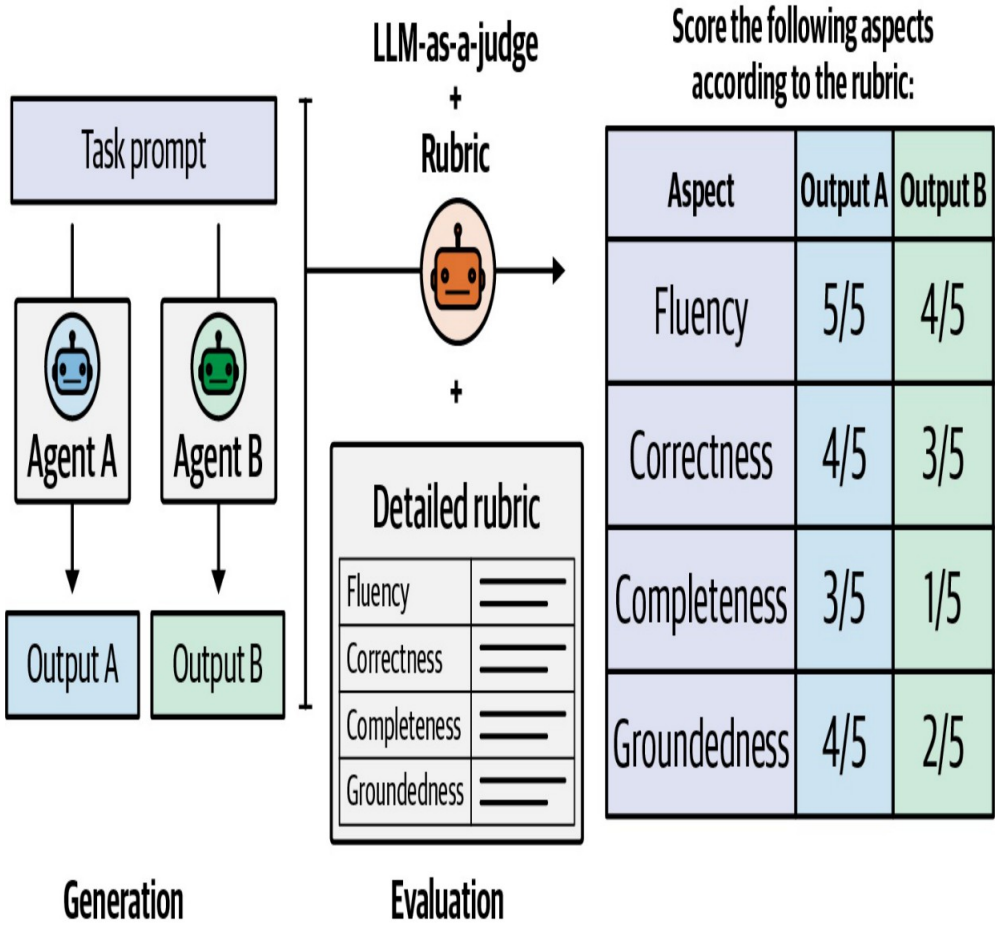
A rubric gives the judge criteria, each with defined score levels, instead of one question such as whether the output is good.

Four axes

The figure scores outputs on fluency, correctness, completeness, and groundedness, each out of five, for Agent A and Agent B.

Examples per level

Rubric levels have descriptions, for example from "severely lacking" up to "comprehensive". ScholarQABench and HealthBench use detailed rubrics.



Grootendorst & Alammar (2026). Fig. 7-7, PDF p. 368

Why rubrics improve reliability

A scale with anchors

A rubric item such as a 1 to 3 scale for citing evidence says what each score means: no sources, sources that are not directly relevant, sources that directly support the claim.

The coverage scale

Summarized from ScholarQABench: 1 severely lacking, 2 partial, 3 acceptable, 4 good, 5 comprehensive, with a description of what each level covers.

Smaller, more objective

Rubrics break a complex judgment into smaller, more objective criteria. They also score sub-behaviors, which shows where the agent performs well or breaks down.

Grootendorst & Alammar (2026). PDF pp. 367–369

Trajectory evaluation across many runs

A trajectory

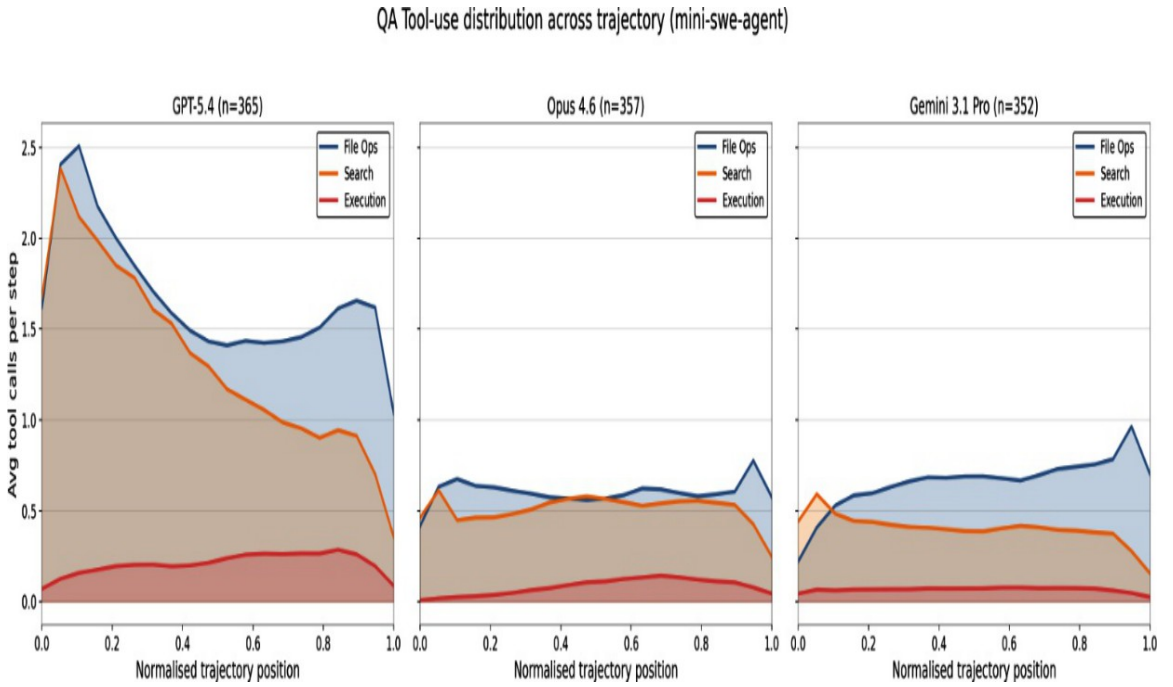
The full sequence of steps: the reasoning produced, the tools called, the arguments passed, and the intermediate outputs along the way.

Tool-use statistics

Looking at tool calls across many runs shows behavior. The figure shows three models on the same tasks, from the SWE Atlas paper.

Patterns outcomes miss

One model searches and does file operations early, and then executes after gathering enough context about the repository.



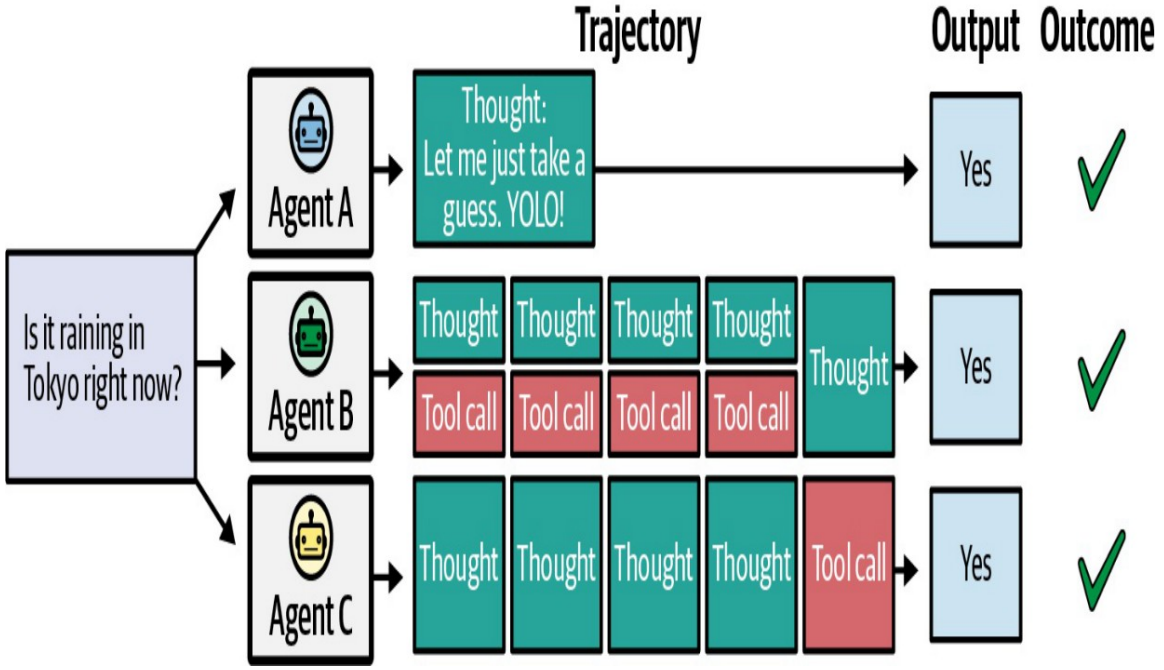
Grootendorst & Alammar (2026). Fig. 7-8, PDF p. 369

Three correct outcomes, three different problems

Agent A guessed
It answered without using a tool, and it was lucky. The thought says to just take a guess.

Agent B was inefficient
It spent a large number of steps and tool calls where one call would have been enough.

Agent C overthought
It made the right tool call but after too many thoughts. Outcome evaluation passes all three.



Grootendorst & Alammar (2026). Fig. 7-9, PDF p. 370

How to measure a trajectory

Score the steps
Measuring a trajectory means scoring the steps along the way, not only the endpoint.

Right tools, valid arguments?
Did the agent reach for the right tools and pass them valid arguments?

How efficient?
How efficient was it in tool calls, steps, and reasoning tokens? Did each step follow from what it learned before?

A judge with the trajectory
An LLM judge can be handed the full trajectory. That is a rubric that examines steps. T-Eval, AgentBoard, TRACE, and AgentProcessBench formalize this.

Reliability: capability versus consistency

Stochastic outputs

The same input can produce different outputs across runs. A single-run score can flatter or unfairly penalize an agent.

Run multiple trials

A better picture comes from running the same task several times and measuring how often the agent succeeds.

Two readings

Capability: can the agent succeed at all? Reliability: does it succeed every time? pass@k measures the first, and pass^k the second.

pass@k with two samples

Sample more than once

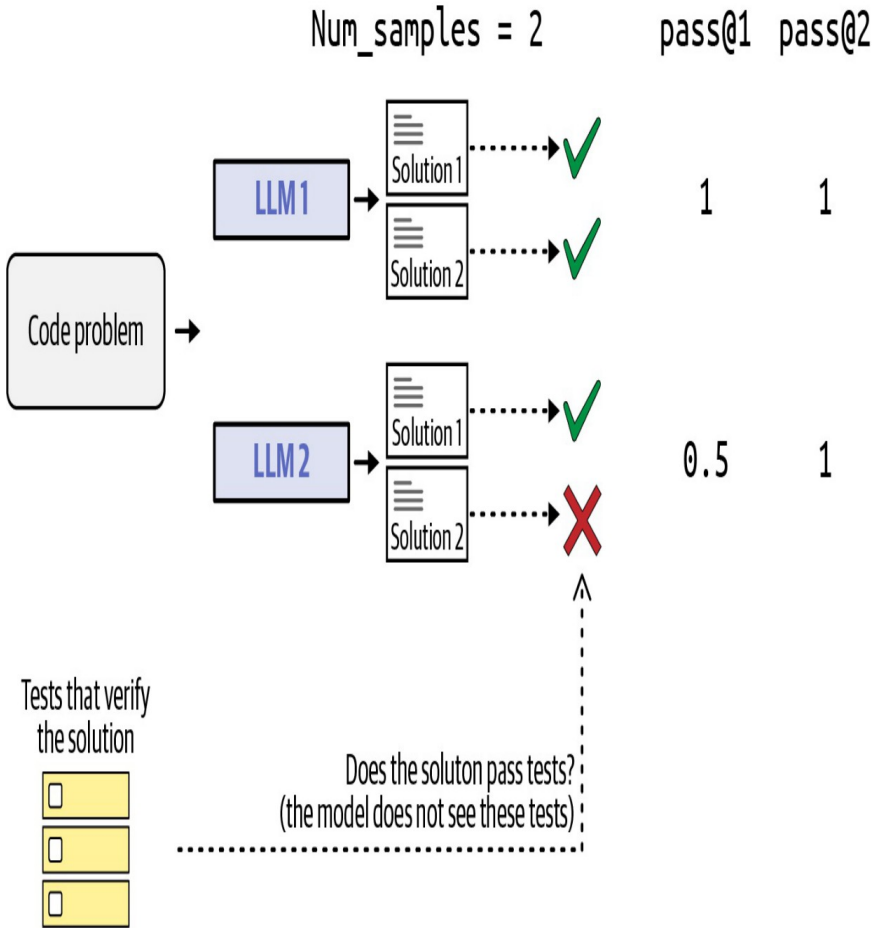
With one sample a model might just be lucky. Sampling twice per model, at a temperature such as 0.7, gives more data to compare.

pass@1

Divide the number of correct answers by the number of samples. LLM 1 passes both, 2/2 = 1. LLM 2 passes one, 1/2 = 0.5.

pass@2 can mislead

Both models score 1 on pass@2, since each has at least one correct sample. With only two samples at k = 2 the estimate is inflated.



Grootendorst & Alammar (2026). Fig. 7-10, PDF p. 372

pass@k with one hundred samples

Scale the samples

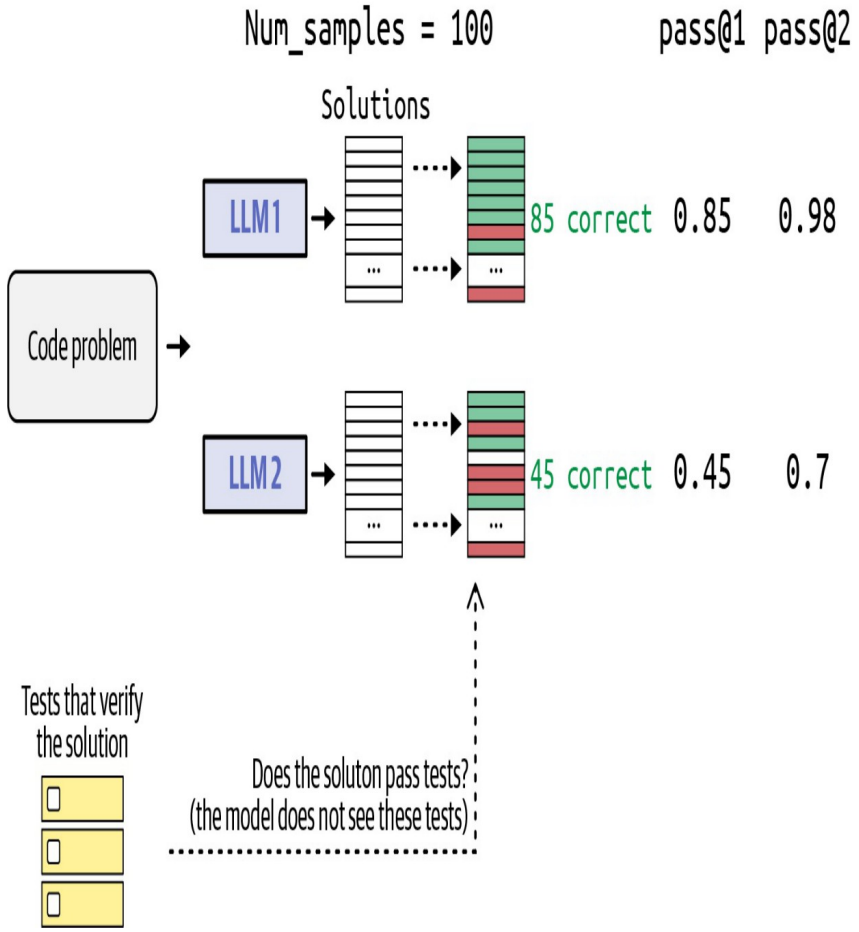
With 100 samples per model the estimates become far more trustworthy. Higher values of k such as 4, 8, 16, and 32 also become possible.

LLM 1

It passes 85 of 100 samples, a pass@1 of 0.85 and a pass@2 of 0.98.

LLM 2

It passes 45 of 100, a pass@1 of 0.45 and a pass@2 of 0.70. The gap that was hidden at two samples is now clear.



Grootendorst & Alammar (2026). Fig. 7-11, PDF p. 373

Computing pass@k

Not simple division

For k above 1, simple division assumes the samples are independent and breaks down. A more robust estimator accounts for the dependency when drawing k candidates.

The estimator

The snippet adapted from Evaluating Large Language Models Trained on Code takes the number of samples n, the number correct c, and k. If fewer than k samples are wrong it returns 1. Otherwise it returns 1 minus a product of terms.

What it estimates

The chance that at least one of k samples drawn from your n is correct. For k = 1 it reduces to simple division, so c divided by n.

Grootendorst & Alammar (2026). Code, PDF pp. 373–374

Measuring reliability with pass^k

The question

pass@k rewards succeeding at least once in k tries, so it rises as you allow more attempts. That is capability. Reliability asks if it succeeds every single time.

The metric

pass^k is the probability that all k sampled attempts succeed. The same runs yield both metrics, and they can tell different stories.

The estimator

From c correct out of n, count the all-correct k-subsets over all k-subsets: C(c, k) divided by C(n, k). It is zero when c is less than k.

Origin

The estimator comes from τ-bench, which introduced pass^k to see whether tool-using agents hold up across repeated trials.

Grootendorst & Alammar (2026). Code, PDF pp. 374–375

pass^3, pass@1, and pass@3 on real models

Same pass@1, different reliability

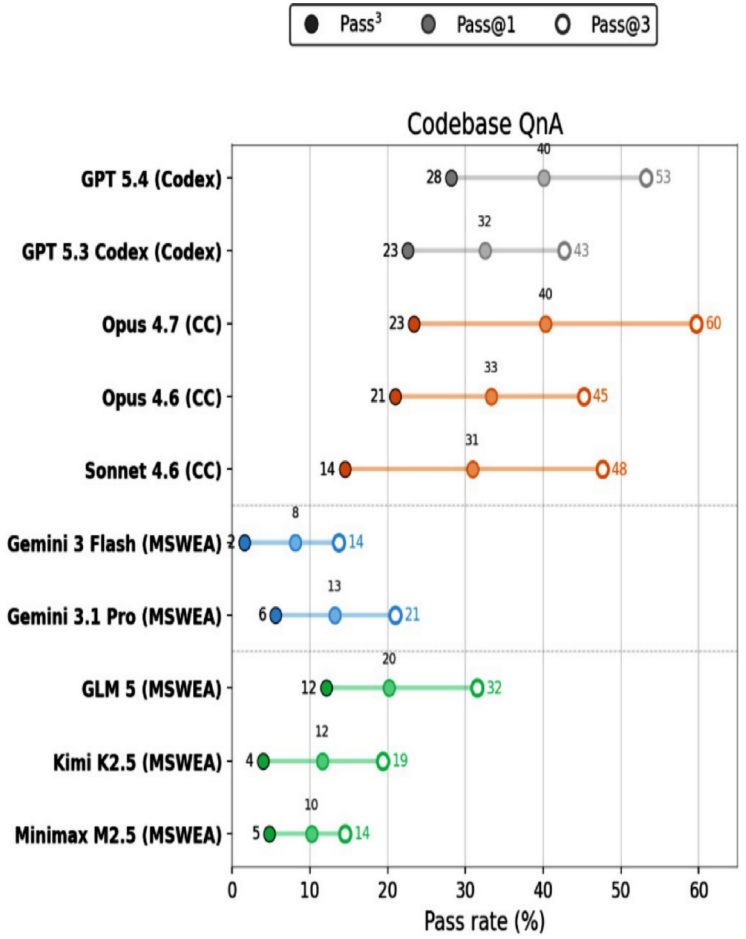
GPT-5.4 and Opus 4.7 both have a pass@1 of 40. But GPT-5.4 has a pass^3 of 28 and Opus 4.7 of 23, so GPT-5.4 is more reliable.

The highest ceiling

Opus 4.7 has the highest pass@3, at 60. It lands at least one success in three tries on more tasks than any other model listed.

Which to use

Choosing a checkpoint to train further favors the ceiling (pass@3). Choosing an API vendor favors reliability, since you run the model as it ships.



Grootendorst & Alammar (2026). Fig. 7-12, PDF p. 376

Safety: who is causing the harm?

Why it comes first

Agents are more consequential than LLMs. They can send a high-impact email, delete a file, or drop a database, so safety evaluation belongs in the suite from the start.

Misuse

A malicious user asks for something harmful, and the question is whether the agent refuses. AgentHarm measures a refusal rate and a harm score.

Manipulation through data

Prompt injection plants instructions in a document, tool result, or web page. Memory poisoning corrupts what is stored. AgentDojo and ASB report an attack success rate.

No adversary

On a benign task the agent harms the user through its own error, such as deleting the wrong files. This is the least standardized and relies on your own tests.

Building your own evals

Representative cases

The best signal about your target task is an evaluation built on your own problem. A small, well-curated set of real usage and data is enough to start.

What it is for

It helps you pick the best LLM and agent framework, and catch regressions when you update any part of the system.

Include failures

Cover a wide set of failure modes and edge cases, not just successes. Score with exact match, programmatic checks, LLM judges, and rubrics.

Start small

Even 5 or 10 cases can give eye-opening signals. Getting started matters more than getting it right immediately.

The eval harness

What it does

An eval harness is the infrastructure that takes test cases, runs the agent on each one, captures the full trajectory, applies your scoring methods, and aggregates the results.

harbor

A recent open source harness designed for agents. It runs trials in containerized environments at scale and uses a standard format for defining tasks and their success criteria.

Run established and custom

Popular benchmarks such as Terminal-Bench ship through the harbor registry, so you can run them alongside your own custom evaluation suite.

Choosing a scoring method

Exact match

Use when there is one right short answer, such as a total or a letter. Pin down the format first. Fails for open-ended output.

Programmatic check

Use when properties can be verified by code, such as valid JSON, tests passing, or word limits. Fast and objective, but only as good as the checks.

LLM judge or rubric

Use for open-ended quality. A rubric adds structure. Use a different model family, swap order, and spot-check against humans.

Human evaluation

Use as the baseline and for the hardest or highest-stakes cases. Slow and costly, so sample it rather than apply it to everything.

Teaching addition. Concepts: Ch. 7, PDF pp. 355–369

Worked example: pass@k versus pass^k

The setup
An agent is run 10 times on one task and 6 runs pass. So n = 10 and c = 6.

pass@1
6 / 10 = 0.60. On a single attempt the agent succeeds 60% of the time.

pass@3
1 minus C(4, 3) / C(10, 3) = 1 − 4/120, about 0.97. Given three tries, it almost always succeeds at least once.

pass^3
C(6, 3) / C(10, 3) = 20/120, about 0.17. All three of three attempts pass only about one time in six.

Activity: critique a benchmark claim

The claim

A vendor says its agent scores 75% on a coding benchmark and is therefore ready for your department's software projects.

The questions

In groups, choose five of the chapter's questions to ask: how was it produced, partial credit, which harness, single or multiple runs, contamination, who ran it, cost, and saturation.

The output

Write the five questions, say what answer would increase your trust, and say what small test of your own would settle the doubt.

Teaching addition. Concepts: Ch. 7, PDF pp. 348–354

Evaluation as a governance practice

Evidence for decisions

Approving or rejecting an agent for use needs evidence. Benchmarks and custom evals supply it, if they are read critically.

Trajectories as records

Captured trajectories show how a result was reached, which supports review, audit, and the investigation of incidents.

Safety testing

Misuse, injection, and benign-error tests give a basis for deciding what an agent may be allowed to do without approval.

Regression checks

Running the suite on every change shows when an update has degraded behavior, which is a basic form of ongoing assurance.

Learning check with worked answers

1. Outcome or trajectory?
Outcome evaluation checks the final result. Trajectory evaluation checks the steps. A lucky guess and an efficient solution can have the same outcome.

2. Why prefer pairwise judgments?
Relative judgments are more reliable. It is easier to say A is better than B than to give each an absolute score.

3. Name two judge biases.
Preference for confident-sounding answers, position bias, and self-preference for the judge's own model family. Mitigate with different families, swapped order, and ensembles.

4. pass@k or pass^k?
pass@k estimates capability, at least one success in k tries. pass^k estimates reliability, success in all k tries.

Teaching addition. Concepts: Ch. 7. Cross-reference: Ch. 5, 6, 10

Chapter summary

Benchmarks and scores

Benchmarks are the artifacts most often met. Read them critically: data contamination, prompt tuning, cost, and saturation all matter. Human evaluation remains the baseline for validating automated methods.

Methods and properties

Automated evaluation runs from exact match and programmatic checks to LLM judges and rubrics. Trajectory evaluation catches unsound reasoning, inefficiency, and overhead. pass@k and pass^k separate capability from reliability.

Safety and your own evals

Safety is organized by who causes the harm. A small, representative suite scored with these methods and run on every change beats waiting for a perfect suite. Part II covers multi-agent systems, multimodal agents, and code agents.

Core distinctions and source guide

Conceptual distinctions

A benchmark scores tasks against a fixed yardstick. Outcome evaluation judges the final result and trajectory evaluation judges the steps. pass@k measures capability, while pass^k measures reliability. Safety is organized by who, if anyone, is causing the harm.

Primary source

Maarten Grootendorst and Jay Alammar. An Illustrated Guide to AI Agents. O'Reilly, first edition, 2026. Core coverage: Chapter 7. Selected connections: Chapters 3, 5, 6, 8, and 10.

Reading the attributions

Figure numbers identify original illustrations. PDF page numbers count the cover as page 1. Source details and further explanation appear in speaker notes. Labeled teaching additions are instructor-created examples and activities.

Source: Supplied 584-page PDF